Validate Three Nodes

AlgoExpert · Hard · Binary Search Trees

The problem

- Is nodeTwo a descendant of one of nodeOne/nodeThree and an ancestor of the other?

Intuition

- Node two must sit on the path between the other two, so search down from node one and node three.

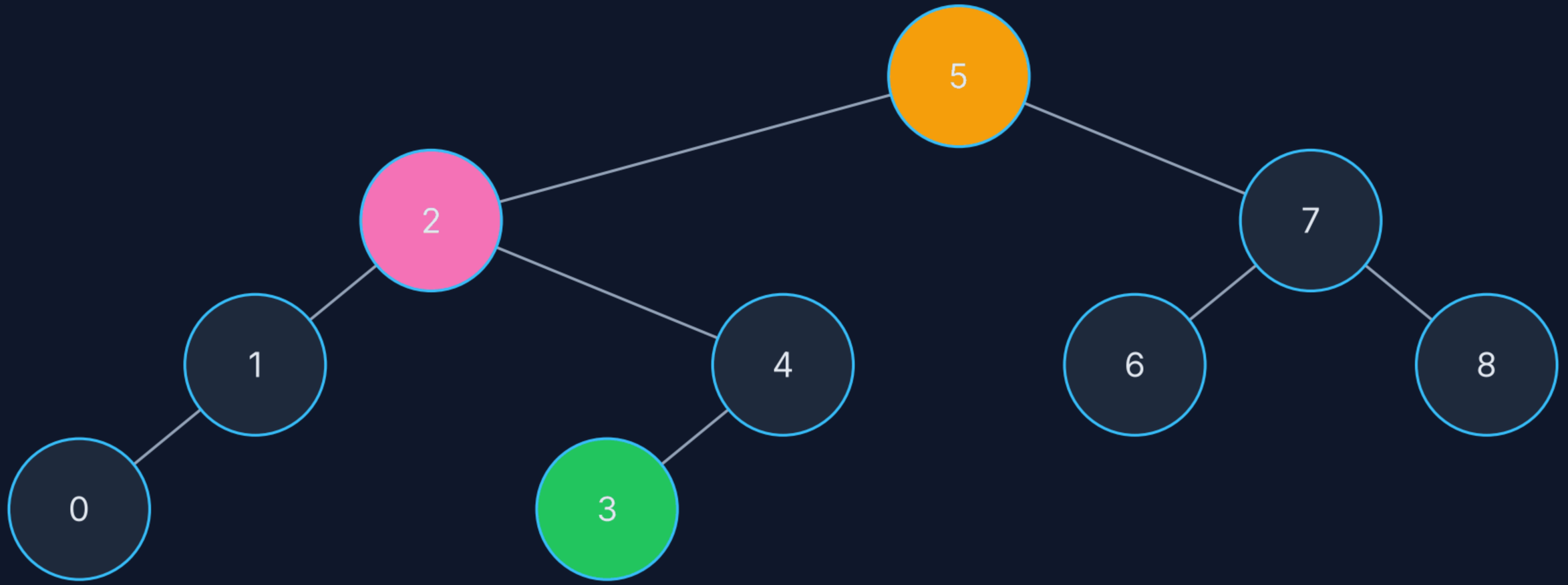
Brute force vs optimal

- Find all paths from root, compare → O(n)
- BST searches between nodes → O(h)
- Search simultaneously from one and three for O(d) best case

Approach

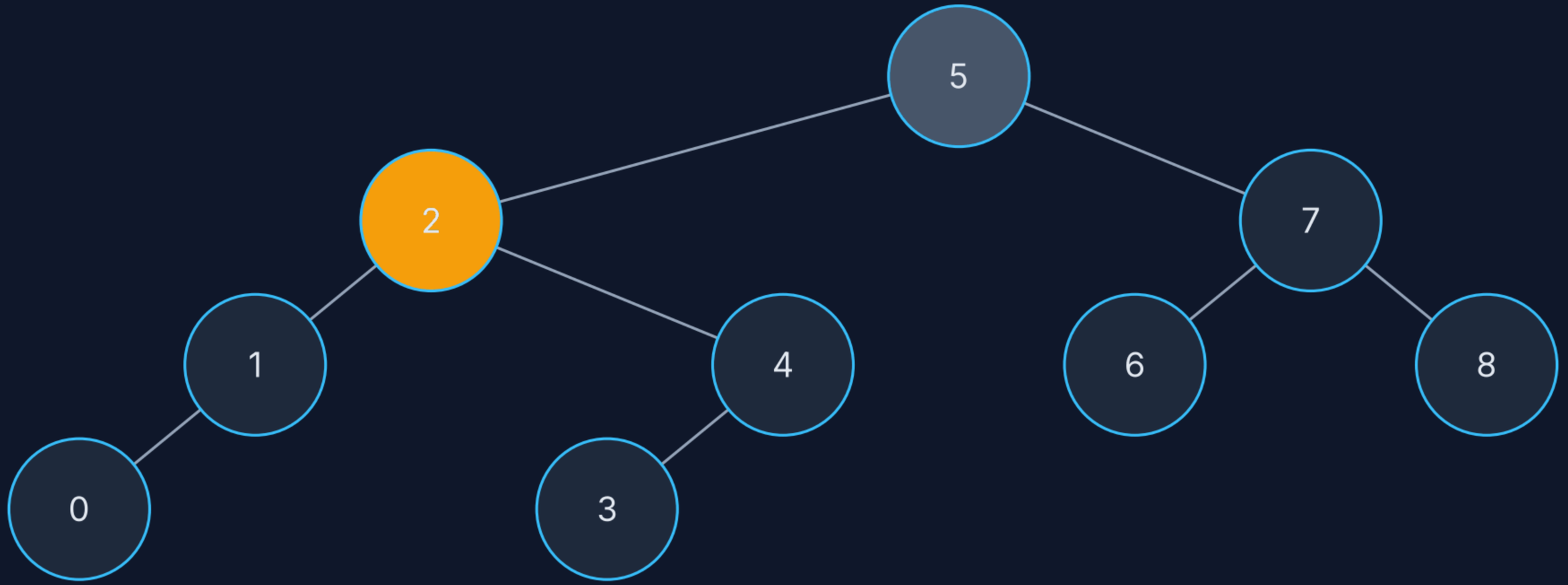
- 1. Check if node two descends from node one or node three
- 2. Then check that the other one descends from node two
- 3. Use BST search for each step

Worked example



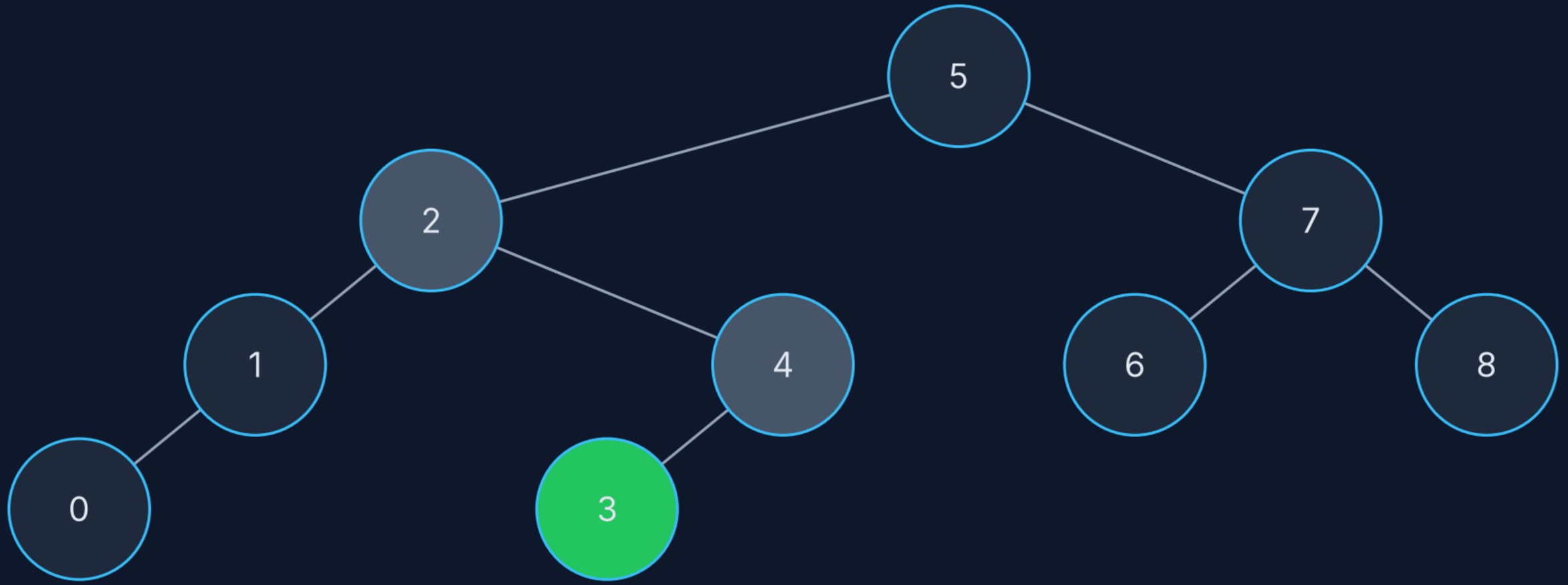
nodeOne = 5, nodeTwo = 2, nodeThree = 3

Worked example



Search 2 from 5: 5 → 2 ✓

Worked example



Search 3 from 2: 2 → 4 → 3 ✓ → true

Java solution

```java
public class ValidateThreeNodes {
    public static boolean validateThreeNodes(BST one, BST two, BST three) {
        if (descends(two, one)) return descends(three, two);
        if (descends(two, three)) return descends(one, two);
        return false;
    }

    /** true if `target` is reachable going down from `from`. */
    private static boolean descends(BST from, BST target) {
        BST c = from;
        while (c != null && c != target) c = target.value < c.value ? c.left : c.right;
        return c == target;
    }
}
```

Complexity

- Time: O(h)
- Space: O(1)

Edge cases & pitfalls

- Either direction: one→two→three or three→two→one
- Compare node identity
- Node two must be strictly between

Interview tips & follow-ups

- Mention the simultaneous search optimisation

Recap

- Node two must sit on the path between the other two, so search down from node one and node three.
- Complexity: O(h)